华清远见 HQYJ.COM

1. 案例代码

1.1 代码步骤

1. 获取数据集
2. 数据基本处理(该案例中省略此步骤)
3. 特征工程(该案例中省略此步骤)
4. 机器学习
5. 模型评估(该案例中省略此步骤)

1.2 代码实现

```
# 1. 导入模块
from sklearn.neighbors import KNeighborsClassifier

# 2. 构造数据集
x = [[1], [2], [3], [4], [5], [16], [17], [18], [19], [20]]
y = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

# 3. 机器学习模型训练
# 实例化一个训练模型
estimator = KNeighborsClassifier(n_neighbors=3)

# 使用fit方法进行训练
estimator.fit(x, y)

# 预测的数据，参数是一个特征值
retVal = estimator.predict([[11]])

# 输出预测的结果
print(retVal)
```

1.3 k-近邻算法API

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *,
weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski',
metric_params=None, n_jobs=None)
    参数说明：

    @ n_neighbors: int, 可选参数(默认为 5)用于kneighbors查询的默认邻居的数量

    @ weights（权重）: str or callable(自定义类型), 可选参数(默认为 ‘uniform’)

        用于预测的权重函数。可选参数如下：

            ‘uniform’ : 统一的权重. 在每一个邻居区域里的点的权重都是一样的。
            ‘distance’ : 权重点等于他们距离的倒数。使用此函数，更近的邻居对于所预测的点的影
响更大。
            [callable] : 一个用户自定义的方法，此方法接收一个距离的数组，然后返回一个相同形
状并且包含权重的数组。
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
    @ algorithm（算法）: {'auto', 'ball_tree', 'kd_tree', 'brute'}, 可选参数（默认为 'auto'）

        计算最近邻居用的算法:
            'ball_tree' 是为了克服kd树高纬失效而发明的，其构造过程是以质心C和半径r分割样本空间，每个节点是一个超球体。
            'kd_tree' 构造kd树存储数据以便对其进行快速检索的树形数据结构，kd树也就是数据结构中的二叉树。以中值切分构造的树，每个结点是一个超矩形，在维数小于20时效率高。
            'brute' 使用暴力搜索.也就是线性扫描，当训练集很大时，计算非常耗时
            'auto' 会基于传入fit方法的内容，选择最合适的算法。

    @ leaf_size（叶子数量）: int, 可选参数(默认为 30)

        传入BallTree或者KDTree算法的叶子数量。此参数会影响构建、查询BallTree或者KDTree的速度，以及存储BallTree或者KDTree所需要的内存大小。 此可选参数根据是否是问题所需选择性使用。

    @ p: integer, 可选参数(默认为 2)

        用于Minkowski metric（闵可夫斯基空间）的超参数。p = 1, 相当于使用曼哈顿距离 (l1), p = 2, 相当于使用欧几里得距离(l2) 对于任何 p ，使用的是闵可夫斯基空间(l_p)

    @ metric（矩阵）: string or callable, 默认为 'minkowski'

        用于树的距离矩阵。默认为闵可夫斯基空间，如果和p=2一块使用相当于使用标准欧几里得矩阵. 所有可用的矩阵列表请查询 DistanceMetric 的文档。

    @ metric_params（矩阵参数）: dict, 可选参数(默认为 None)

        给矩阵方法使用的其他的关键词参数。

    @ n_jobs: int, 可选参数(默认为 1)

        用于搜索邻居的，可并行运行的任务数量。如果为-1, 任务数量设置为CPU核的数量。不会影响fit方法。
```

```
#  方法:
fit(X, y)     使用X作为训练数据，y作为目标值（类似于标签）来拟合模型。
get_params([deep])  获取估值器的参数。
kneighbors([X, n_neighbors, return_distance])    查找一个或几个点的K个邻居。
kneighbors_graph([X, n_neighbors, mode])     计算在X数组中每个点的k邻居的（权重）图。
predict(X)  给提供的数据预测对应的标签。
predict_proba(X)     返回测试数据X的概率估值。
score(X, y[, sample_weight])     返回给定测试数据和标签的平均准确值。
set_params(**params)     设置估值器的参数。
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
fit(X,y):

    使用X作为训练数据，y作为目标值（标签）拟合模型
参数:

    @ X: {类似数组， 稀疏矩阵， BallTree， KDTree}

        待训练数据。如果是数组或者矩阵，形状为 [n_samples， n_features]，如果矩阵
为'precomputed'， 则形状为[n_samples， n_samples]。

    @ y: {类似数组， 稀疏矩阵}
        形状为[n_samples] 或者 [n_samples， n_outputs]的目标值。
```

```
get_params(deep=True)

    获取估值器的参数.
参数:

    @ deep: boolean， 可选参数

        如果为 True， 则返回估值器的参数，以及包含子目标的估值器。

返回值:

    @ params: Mapping string to any

        返回Map变量，内容为[参数值: 值， 参数值: 值， …]。
```

```
kneighbors(X=None,n_neighbors=None,return_distance=True)[source]

    查询一个或几个点的K个邻居， 返回每个点的下标和到邻居的距离。
    （重点使用）

参数:

    @ X: 类似数组， 形状(n_query， n_features)或者(n_query， n_indexed) 。

        如果矩阵是'precomputed'，形状为(n_query， n_indexed)带查询的一个或几个点。如果没有
提供，则返回每个有下标的点的邻居们。

    @ n_neighbors: int

        邻居数量 (默认为调用构造器时设定的n_neighboes的值).

    @ return_distance: boolean， 可选参数. 默认为 True.

        如果为 False，则不会返回距离

返回值:

dist: array

    当return_distance =True时，返回到每个点的长度。
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
ind: array

    邻居区域里最近的几个点的下标。
```

```
kneighbors_graph(X=None,n_neighbors=None,mode='connectivity')[source]

    计算在X数组中每个点的k邻居的（权重）

参数:

    @ X: 类似数组, 形状(n_query, n_features)。

        如果矩阵是'precomputed', 形状为(n_query, n_indexed)一个或多个待查询点。如果没有提供，则返回每个有下标的点的邻居们。

    @ n_neighbors: int

        邻居数量。(默认为调用构造器时设定的n_neighboes的值)。

    @ mode: {'connectivity', 'distance'}, 可选参数

        返回矩阵数据类型: 'connectivity' 会返回1和0组成的矩阵。 in 'distance' 会返回点之间的欧几里得距离。

返回值:

    @ A: CSR格式的稀疏矩阵，形状为 [n_samples, n_samples_fit]

    @ n_samples_fit 是拟合过的数据中样例的数量，其中 A[i, j] 代表i到j的边的权重。
```

```
predict(X)[source]

给提供的数据预测相应的类别标签

参数:

    @ X: 类似数组, 形状(n_query, n_features)。

        如果矩阵是'precomputed', 形状为(n_query, n_indexed) 待测试样例。

返回值:

    @ y: 形状为 [n_samples] 或者 [n_samples, n_outputs]的数组

        返回每个待测试样例的类别标签。
```

```
predict_proba(X)[source]

    返回测试数据X的概率估值。
参数:

    @ X: 类似数组, 形状(n_query, n_features)。

        如果矩阵是'precomputed', 形状为(n_query, n_indexed)待测试样例。
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
返回值:

    @ p: 形状为[n_samples, n_classes]的数组，或者是n_outputs列表

        输入样例的类别概率估值。其中类别根据词典顺序排序。
```

```
score(X, y, sample_weight=None)[source]

    返回给定测试数据和标签的平均准确度。在多标签分类中，返回的是各个子集的准确度。

参数:

    @ X : 类似数组，

        形状为 (n_samples, n_features)待测试样例

    @ y: 类似数组，

        形状为 (n_samples) 或者 (n_samples, n_outputs)X对于的正确标签

    @ sample_weight: 类似数组，

        形状为 [n_samples], 可选参数待测试的权重

返回值:

    @ score : float

        self.predict(X) 关于y的平均准确率。
```

```
set_params(**params)[source]

    设置估值器的参数。
    此方法在单个估值器和嵌套对象（例如管道）中有效。而嵌套对象有着__形式的参数，方便更新各个参数。
```

华清远见 fsdev.com.cn

2. KNN算法实现鸢尾花数据集分类

2.1 数据集介绍

iris数据集是常用的分类实验数据集，由Fisher在1936收集整理。Iris也称为鸢尾花卉数据集，是一类多重变量分析的数据集。

华清远见 HQYJ.COM

Iris鸢尾花数据集： 包含3个目标值分别为：

山鸢尾（Iris-setosa）；

变色鸢尾（Iris-versicolor）；

尼亚鸢尾（Iris-virginica），

包含4个特征值：

Sepal.Length（花萼长度），单位是cm;

Sepal.Width（花萼宽度），单位是cm;

Petal.Length（花瓣长度），单位是cm;

Petal.Width（花瓣宽度），单位是cm;

共 150 条数据，每类各 50 个数据，每条记录都有 4 项特征：花萼长度、花萼宽度、花瓣长度、花瓣宽度，通常可以通过这4个特征预测鸢尾花卉属于哪一品种。

2.2 Scikit-Learn中的数据集介绍

2.2.1 Scikit-Learn数据集API介绍

```
sklearn.datasets
    加载获取流行数据集
    datasets.load_iris()
        获取小规模数据集，数据包含周期datasets中

    datasets.fetch_20newsgroups(data_home=None)
        获取大规模数据集，需要从网络下载，函数的第一个参数是data_home,表示数据集下载的目录，
默认为~/scikit_learn_data
```

```
    sklearn 小规模数据集
        datasets.load_iris()
        加载并返回鸢尾花数据集
```

```
    # 鸢尾花数据集获取

    from sklearn.datasets import load_iris, fetch_20newsgroups

    # 小规模数据集获取
    iris = load_iris()

    print(iris)
```

2.2.2 sklearn数据返回值介绍

```
load_iris返回的数据类型datasets.base.BUnch（字典格式）
    data : 特征数据数组，是[n_samples * n_features]的二维numpy.ndarray数组
    target :  标签数组，是n_samples的一维numpy.ndarray数组
    DESCR :  数据描述
    feature_names : 特征名，  新闻数据，手写数字，回归数据没有
    target_name : 标签名(目标明)
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
# 鸢尾花数据集获取

from sklearn.datasets import load_iris, fetch_20newsgroups

# 小规模数据集获取
iris = load_iris()

# print("鸢尾花数据值的返回值 :  \n", iris)

# 返回值是一个集成自字典的Bench
print("鸢尾花的特征值 : \n", iris["data"])
print("鸢尾花的目标值 : \n", iris.target)
print("鸢尾花特征的名字 : \n", iris.feature_names)
print("鸢尾花目标值的名字 : \n", iris.target_names)
print("鸢尾花的描述 : \n", iris.DESCR)
```

2.3 鸢尾花数据可视化

数据可视化在数据挖掘中是一个很重要的部分，将数据用图表形式展示可以很直观地看到数据集的特点（比如正态分布，长尾分布，聚集等），方便下一步怎么对数据进行处理。

seaborn框架是一个广受欢迎的可视化框架，提到seaborn不得不提到的还有matplotlib，matplotlib是一个强大的科学绘图包，里面集成了大量可视化图表，但是参数比较多，使用起来比较繁琐，而seaborn对这方面做了优化，不过seaborn不是matplotlib的一个替代，而是一个补充。它们的官网分别如下：

seaborn官网：http://seaborn.pydata.org/index.html
matplotlib官网：https://matplotlib.org/2.2.2/index.html

至于seaborn可以画哪些图，在seaborn的官网上有一个gallery，专门展示它的图表示例。
http://seaborn.pydata.org/examples/index.html

```
    基础绘图

    seaborn一共有5个大类21种图，分别是:

        Relational plots 关系类图表
            1. relplot() 关系类图表的接口，其实是下面两种图的集成，通过指定kind参数可以画出下面的两种图
            2. scatterplot() 散点图
            3. lineplot() 折线图
        Categorical plots 分类图表
            1. catplot() 分类图表的接口，其实是下面八种图表的集成，，通过指定kind参数可以画出下面的八种图
            2. stripplot() 分类散点图
            3. swarmplot() 能够显示分布密度的分类散点图
            4. boxplot() 箱图
            5. violinplot() 小提琴图
            6. boxenplot() 增强箱图
            7. pointplot() 点图
            8. barplot() 条形图
            9. countplot() 计数图
        Distribution plot 分布图
            1. jointplot() 双变量关系图
            2. pairplot() 变量关系组图
            3. distplot() 直方图，质量估计图
            4. kdeplot() 核函数密度估计图
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
            5. rugplot() 将数组中的数据点绘制为轴上的数据
        Regression plots 回归图
            1. lmplot() 回归模型图
            2. regplot() 线性回归图
            3. residplot() 线性回归残差图
        Matrix plots 矩阵图
            1. heatmap() 热力图
            2. clustermap() 聚集图
```

```python
# coding:utf-8

# 内嵌绘图
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.datasets import load_iris, fetch_20newsgroups

# 1. 小规模数据集获取
iris = load_iris()

# print("鸢尾花数据值的返回值 :  \n", iris)

# 2. 数据属性描述
# 返回值是一个集成自字典的Bench
# print("鸢尾花的特征值 : \n", iris["data"])
# print("鸢尾花的目标值 : \n", iris.target)
# print("鸢尾花特征的名字 : \n", iris.feature_names)
# print("鸢尾花目标值的名字 : \n", iris.target_names)
# print("鸢尾花的描述 : \n", iris.DESCR)

# 3. 数据可视化
# 3.1 数据类型转换，把数据用DataFrame存储
iris_data = pd.DataFrame(data=iris.data, columns = ['Sepal_Length',
'Sepal_Width', 'Petal_Length', 'Petal_Width'])
# 填充目标值
iris_data["target"] = iris.target
# print(iris_data)

def plot_iris(data, col1, col2):
    # x,  y : x轴和y轴数据索引值
    # data : 具体的数据
    # hue : 目标值，可以通过不同的目标先是不同的颜色
    # fit_reg : 是否进行线性拟合

    sns.lmplot(x=col1, y=col2, data=data, hue="target",fit_reg=False)
    plt.title("鸢尾花数据展示")
    plt.xlabel(col1)
    plt.ylabel(col2)
    plt.show()


plot_iris(iris_data, 'Sepal_Length', 'Petal_Width')
```

2.4 数据集划分

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
机器学习一般的数据集划分分为两部分
    1. 训练数据：用于训练，构建模型
    2. 测试数据：在模型校验时使用，用于评估模型是否有效
划分比例：
    1. 训练集 ：  70%    80%    75%
    2. 测试集 ：  30%    20%    25%
```

```
    数据集划分API
        sklearn.model_selection.train_test_split(arrays, *options)
            x ：  数据集的特征值
            y ：  数据集的特征值
            test_size ： 测试卷的大小，  一般为float
            random_state ： 随机数种子，不同的种子会造成不同的随机采样结果。相同的种子采样结果相同。
            return ： 测试机特征训练集特征值，训练标签，测试标签(默认随机取)
```

```
    # coding:utf-8

# 内嵌绘图
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd


from sklearn.datasets import load_iris, fetch_20newsgroups
from sklearn.model_selection import train_test_split
# 1. 小规模数据集获取
iris = load_iris()

# print("鸢尾花数据值的返回值 ： \n", iris)

# 2. 数据属性描述
# 返回值是一个集成自字典的Bench
# print("鸢尾花的特征值 ： \n", iris["data"])
# print("鸢尾花的目标值 ： \n", iris.target)
# print("鸢尾花特征的名字 ： \n", iris.feature_names)
# print("鸢尾花目标值的名字 ： \n", iris.target_names)
# print("鸢尾花的描述 ： \n", iris.DESCR)

# 3. 数据可视化
# 3.1 数据类型转换，把数据用DataFrame存储
iris_data = pd.DataFrame(data=iris.data, columns = ['Sepal_Length',
'Sepal_Width', 'Petal_Length', 'Petal_Width'])
# 填充目标值
iris_data["target"] = iris.target
# print(iris_data)

def plot_iris(data, col1, col2):
    # x,  y ： x轴和y轴数据索引值
    # data ： 具体的数据
    # hue ： 目标值，可以通过不同的目标先是不同的颜色
    # fit_reg ： 是否进行线性拟合

    sns.lmplot(x=col1, y=col2, data=data, hue="target",fit_reg=False)
    plt.title("鸢尾花数据展示")
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
    plt.xlabel(col1)
    plt.ylabel(col2)
    plt.show()


# plot_iris(iris_data, 'Sepal_Length', 'Sepal_Width')
# plot_iris(iris_data, 'Sepal_Length', 'Petal_Length')
# plot_iris(iris_data, 'Sepal_Length', 'Petal_Width')
# plot_iris(iris_data, 'Sepal_Width', 'Petal_Length')
# plot_iris(iris_data, 'Sepal_Width', 'Petal_Width')
# plot_iris(iris_data, 'Petal_Length', 'Petal_Width')

# 4. 数据集划分
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target,
test_size=0.2)

print("训练集的特征值是 :  \n", x_train)
print("测试集的特征值是 :  \n", x_test)
print("训练集的目标值是 :  \n", y_train)
print("测试集的目标值是 :  \n", y_test)

print("训练集的特征值形状 :  \n", x_train.shape)
print("测试集的特征值形状 :  \n", x_test.shape)
print("训练集的目标值形状 :  \n", y_train.shape)
print("测试集的目标值形状 :  \n", y_test.shape)
```

2.5 特征预处理

2.5.1 什么是特征预处理

2.5.1.1 特征预处理定义

通过一些转换函数将特征数据转换成更加适合算法模型的特征数据过程

为什么我们要进行归一化/标准化?

特征的单位或者大小相差较大，或者某特征的方差相比其他的特征要大出几个数量级，容易影响（支配）目标 结果，使得一些算法无法学习到其它的特征

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

举例：约会对象数据

相亲约会对象数据，这个样本时男士的数据，三个特征，玩游戏所消耗时间的百分比、每年获得的飞行常客里程数、每周消费的冰淇淋公升数。然后有一个所属类别，被女士评价的三个类别，不喜欢didnt、魅力一般small、极具魅力large也许也就是说飞行里程数对于结算结果或者说相亲结果影响较大，但是统计的人觉得这三个特征同等重要。

里程数	公升数	消耗时间比	评价
14488	7.153469	1.673904	smallDoses
26052	1.441871	0.805124	didntLike
75136	13.147394	0.428966	didntLike
38344	1.669788	0.134296	didntLike
72993	10.141740	1.032955	didntLike
35948	6.830792	1.213192	largeDoses
42666	13.276369	0.543880	largeDoses
67497	8.631577	0.749278	didntLike
35483	12.273169	1.508053	largeDoses
50242	3.723498	0.831917	didntLike

我们需要用到一些方法进行无量纲化，使不同规格的数据转换到同一规格

2.5.1.2 包含内容(数值型数据的无量纲化)

归一化

标准化

2.5.1.3 特征预处理API

sklearn.preprocessing

2.5.2 归一化

2.5.2.1 定义

通过对原始数据进行变换把数据映射到(默认为[0,1])之间

2.5.2.2 公式

$$X' = \frac{x - min}{max - min} \qquad X'' = X' * (mx - mi) + mi$$

作用于每一列，max为一列的最大值，min为一列的最小值,那么X”为最终结果，mx，mi分别为指定区间值默认 mx为1,mi为0

特征1	特征2	特征3	特征4
90	2	10	40
60	4	15	45
75	3	13	46

特征1	特征2	特征3	特征4
$\frac{90-60}{90-60}$	$\frac{2-2}{4-2}$	$\frac{10-10}{15-10}$	$\frac{40-40}{46-40}$
$\frac{60-60}{90-60}$	$\frac{4-2}{4-2}$	$\frac{15-10}{15-10}$	$\frac{45-40}{46-40}$
$\frac{75-60}{90-60}$	$\frac{3-2}{4-2}$	$\frac{13-10}{15-10}$	$\frac{46-40}{46-40}$

注：里面是第一步，还需要第二步乘以(1-0)+0

CSDN @小玩偶啊

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

2.5.2.3 API

```
sklearn.preprocessing.MinMaxScaler (feature_range=(0,1)… )
    MinMaxScalar.fit_transform(X)
        ------X:numpy array格式的数据[n_samples,n_features]
    返回值：转换后的形状相同的array
```

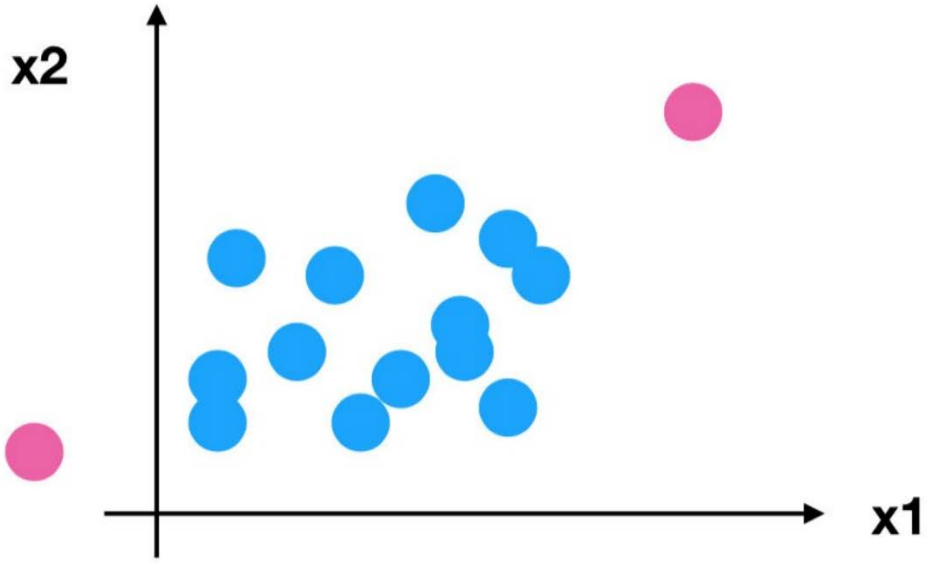
2.5.2.4 数据计算

```
我们对以下数据进行运算，在dating.txt中。保存的就是之前的约会对象数据

    milage,Liters,Consumtime,target
    40920,8.326976,0.953952,3
    14488,7.153469,1.673904,2
    26052,1.441871,0.805124,1
    75136,13.147394,0.428964,1
    38344,1.669788,0.134296,1

1、实例化MinMaxScalar
2、通过fit_transform转换
```

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler,StandardScaler


# 归一化演示

data=pd.read_csv("~/Desktop/data.txt")
# print(data)

# 1. 实例化
transfer=MinMaxScaler(feature_range=(0,1))
# 2. 进行转换，调用fit_transform
ret_data=transfer.fit_transform(data[["milage","Liters","Consumtime"]])
print("归一化处理后的结果\n",ret_data)
```

华清远见 fsdev.com.cn

问题：如果数据中异常点较多，会有什么影响？



CSDN @小玩偶啊

华清远见 HQYJ.COM

2.5.2.5 归一化总结

注意最大值最小值是变化的，另外，最大值与最小值非常容易受异常点影响，所以这种方法鲁棒性较差，只适合传统精 确小数据场景。

2.5.3

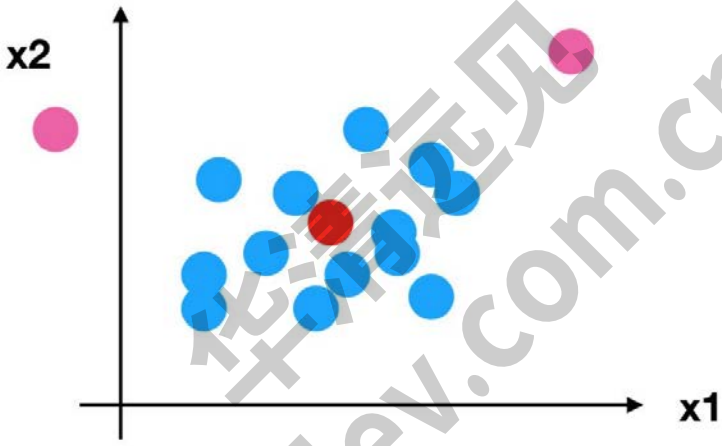
2.5.3.1 定义

通过对原始数据进行变换把数据变换到均值为0,标准差为1范围内

2.5.3.2 公式

$$X' = \frac{x - \text{mean}}{\sigma}$$

作用于每一列，mean为平均值，σ为标准差



CSDN @小玩偶啊

华清远见 fsdev.com.cn

对于归一化来说：如果出现异常点，影响了最大值和最小值，那么结果显然会发生改变

对于标准化来说：如果出现异常点，由于具有一定数据量，少量的异常点对于平均值的影响并不大，从而方差改变 较小

2.5.3.3 API

```
sklearn.preprocessing.StandardScaler( )

处理之后每列来说所有数据都聚集在均值0附近标准差差为1
StandardScaler.fit_transform(X)
    ----- X:numpy array格式的数据[n_samples,n_features]
返回值：转换后的形状相同的array
```

2.5.3.4 数据计算

```
同样对上面的数据进行处理分析

1. 实例化StandardScaler
2. 通过fit_transform转换
```

华清远见 HQYJ.COM

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler,StandardScaler

def stand_demo():
    '''
    标准化演示
    :return:
    '''
    data=pd.read_csv("~/Desktop/data.txt")
   # print(data)
    # 1. 实例化
    transfer=StandardScaler()
    # 2. 进行转换，调用fit_transform
    ret_data=transfer.fit_transform(data[["milage","Liters","Consumtime"]])
    print("标准化处理后的结果\n",ret_data)
    print("每一列的方差为：\n",transfer.var_)
    print("每一列的均值：\n",transfer.mean_)

stand_demo()
```

2.5.3.5 标准化总结

在已有样本足够多的情况下比较稳定，适合现代嘈杂大数据场景。

2.6 案例：鸢尾花种类预测-流程实现

主要目的是对前边内容的复习和总结。

2.6.1 再识K-近邻算法API

```
sklearn.neighbors.KNeighborsClassifier(n_neighbors=5,algorithm='auto')

n_neighbors：
    int,可选（默认= 5），k_neighbors查询默认使用的邻居数
algorithm：{'auto'，'ball_tree'，'kd_tree'，'brute'}
快速k近邻搜索算法，默认参数为auto，可以理解为算法自己决定合适的搜索算法。除此之外，用户也可 以自己指定搜索算法ball_tree、kd_tree、brute方法进行搜索，
    brute是蛮力搜索，也就是线性扫描，当训练集很大时，计算非常耗时。
    kd_tree，构造kd树存储数据以便对其进行快速检索的树形数据结构，kd树也就是数据结构中的二叉树。以中值切分构造的树，每个结点是一个超矩形，在维数小于20时效率高。
    ball tree是为了克服kd树高维失效而发明的，其构造过程是以质心C和半径r分割样本空间，每个节点是一个超球体。
```

2.6.2 案例：鸢尾花种类预测

2.6.2.1 数据集介绍

Iris数据集是常用的分类实验数据集，由Fisher, 1936收集整理。Iris也称鸢尾花卉数据集，是一类多重变量分析的数据 集。关于数据集的具体介绍：

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
Iris鸢尾花数据集:
包含3个目标值分别为:
    山鸢尾（Iris-setosa）;
    变色鸢尾（Iris-versicolor）;
    维吉尼亚鸢尾（Iris-virginica）,

包含4个特征值:
    Sepal.Length（花萼长度）, 单位是cm;
    Sepal.Width（花萼宽度）, 单位是cm;
    Petal.Length（花瓣长度）, 单位是cm;
    Petal.Width（花瓣宽度）, 单位是cm;
共 150 条数据，每类各 50 个数据，每条记录都有 4 项特征：花萼长度、花萼宽度、花瓣长度、花瓣宽度，通常可以通过这4个特征预测鸢尾花卉属于哪一品种。
```

2.6.2.2 步骤分析

```
1.获取数据集
2.数据基本处理
3.特征工程
4.机器学习(模型训练)
5.模型评估
```

2.6.2.3 代码实现

```
#获取数据
from sklearn.datasets import load_iris
#数据基本处理（训练集和测试集）
from sklearn.model_selection import train_test_split
#特征工程和特征预处理
from sklearn.preprocessing import StandardScaler
#机器学习
from sklearn.neighbors import KNeighborsClassifier


# 1.获取数据
iris=load_iris()

# 2.数据基本处理
x_train,x_test,y_train,y_test=train_test_split(iris.data,iris.target,test_size=0
.2,random_state=5)

# 3.特征工程 - 特征预处理
```

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

```
transfer=StandardScaler()
x_train=transfer.fit_transform(x_train)
x_test=transfer.fit_transform(x_test)

# 4.机器学习-KNN
# 4.1 实例化一个估计器
estimator=KNeighborsClassifier(n_neighbors=5)
# 4.2 模型训练
estimator.fit(x_train,y_train)

# 5.模型评估
# 5.1 预测值结果输出
y_pre=estimator.predict(x_test)
print("预测值是:\n",y_pre)
print("预测值和真实值的对比是:\n",y_pre==y_test)

# 5.2 准确率计算
score=estimator.score(x_test,y_test)
print("准确率是: \n",score)
```

使用可视化加载和探索数据，以确定特征是否能将不同类别分开。
通过标准化数字特征并随机抽样到训练集和测试集来准备数据。
通过统计学，精确度度量进行构建和评估机器学习模型。

3. k近邻算法优缺点汇总

3.1 优点

简单有效：重新训练的代价低

适合类域交叉样本：KNN方法主要靠周围有限的邻近的样本,而不是靠判别类域的方法来确定所属类别的，因此对于类域的交 叉或重叠较多的待分样本集来说，KNN方法较其他方法更为适合。

适合大样本自动分类：该算法比较适用于样本容量比较大的类域的自动分类，而那些样本容量较小的类域采用这种算法比较容易 产生误分。

3.2 缺点

惰性学习：KNN算法是懒散学习方法（lazy learning,基本上不学习），一些积极学习的算法要快很多

类别评分不是规格化：
不像一些通过概率评分的分类

输出可解释性不强：例如决策树的输出可解释性就较强

对不均衡的样本不擅长：当样本不平衡时，如一个类的样本容量很大，而其他类样本容量很小时，有可能导致当输入一个新样本 时，该样本的K个邻居中大容量类的样本占多数。该算法只计算“最近的”邻居样本，某一类的样本数量很 大，那么或者这类样本并不接近目标样本，或者这类样本很靠近目标样本。无论怎样，数量并不能影响运 行结果。可以采用权值的方法（和该样本距离小的邻居权值大）来改进。

计算量较大：目前常用的解决方法是事先对已知样本点进行剪辑，事先去除对分类作用不大的样本。

华清远见 fsdev.com.cn

华清远见
HQYJ.COM

华清远见
fsdev.com.cn